

Course Outline: From Raw Data to Reports

Title: From Raw Data to Reports: Transforming What You Already Have **Skill:** Skill 44: Building a report from the telemetry or user data **Learnpack:** + De datos crudos a reportes: transformando lo que ya tienes **File:** s44-w15d44-de-datos-crudos-a-reportes-transformando **Prerequisite:** Skill 43 (Collecting telemetry and context-related info) **Target Audience:** Beginner developers who have collected telemetry data and need to turn it into actionable reports **Total Lessons:** 11 **Format:** Mixed (9% hands-on = 1 lesson)

Course Description

You've been collecting data — user events, TODO completions, telemetry from your app. Now what? Raw data sitting in a CRUD-optimized database can't directly answer questions like "how many tasks did users complete per day this week?" This course teaches you to bridge that gap using pandas: load the raw data, filter it to what matters, group it by time, aggregate it into a number that means something, and serve it as a usable report.

Course Philosophy

This course emphasizes:

- Starting from data you already have — no new collection needed
 - Asking the question first, then shaping the data to answer it
 - Building one complete, runnable pipeline from raw rows to JSON output
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - The Data Gap	2
02 - The Transformation Pipeline	4
03 - Report Outputs	2
04 - Assessment	1
05 - Outro	1
Total	11

00.0 From Raw Data to Reports 

Learning Objectives:

- Recognize when data is being collected but not used
- Understand what "transforming data" means in practical terms
- Preview the five-step pipeline this course builds

Content Outline:

- The gap between storing data and using it
- Why the same data that powers your app can't directly power your reports
- What students will build: a complete pandas pipeline that produces a weekly completion report
- Overview of course sections

Transitions: This lesson frames the problem. Section 01 names it precisely.

Key Principles:

- Data has to be reshaped before it can answer questions
- The transformation work is predictable — it follows the same pattern every time

Examples:

- A TODO app storing completions per user, but the dashboard can't show "completions per day this week"
 - Telemetry events arriving as individual rows, but the product team needs weekly trend lines
-

01.0 The Data Gap

Learning Objectives:

- Distinguish transactional data from analytical data
- Explain why CRUD-optimized storage is not report-ready
- Identify the structural reason aggregation requires transformation

Content Outline:

- Transactional data: designed for creating, reading, updating, deleting individual records
- Analytical data: designed for querying across many records, grouping, and summarizing
- Why these two goals produce different data shapes
- How 3NF (already covered in Skill 40) serves transactional integrity but frustrates analytics

Transitions: This lesson establishes the conceptual gap. The next lesson (01.1) shows it concretely with a TODO table.

Key Principles:

- The same database that's great for writing individual rows is often poor for aggregating across them
- CRUD data is stored for fidelity; analytical data is shaped for questions
- Transformation bridges the two — it doesn't change the source, it reshapes for a different purpose

Examples:

- A `todos` table: each row is one TODO with a `status` and `completed_at`. Perfect for CRUD. Useless as-is for "how many completions per day?"
 - An `events` table: each row is one user action. Perfect for logging. Needs aggregation to become a funnel.
-

01.1 The TODO Problem

Learning Objectives:

- Walk through why a standard TODO table cannot directly answer an analytics question
- Identify what transformation steps would be required to get the answer
- Connect the abstract gap to a concrete, familiar data structure

Content Outline:

- The TODO table schema: `id`, `user_id`, `title`, `status`, `completed_at`
- The question: "How many TODOs did users complete each day this week?"
- Why a simple `SELECT *` or ORM query doesn't answer it
- What needs to happen: filter by status and date → group by day → count per group
- The result shape: `{date: count}` — fundamentally different from the source rows

Transitions: Now that you understand the problem, Section 02 walks through each step of the solution.

Key Principles:

- The answer to an analytics question is never a single row — it's a summary across rows
- The shape of the answer (one number per day) is completely different from the shape of the source (one row per TODO)
- Recognizing this mismatch is the first step to solving it

Examples:

- Raw rows: `{id: 1, status: "completed", completed_at: "2024-01-15"}, {id: 2, status: "pending", ...}, {id: 3, status: "completed", completed_at: "2024-01-15"}, ...`
 - Desired output: `{"2024-01-15": 2, "2024-01-16": 5, "2024-01-17": 3}`
 - The gap: going from ~200 rows to 7 numbers requires transformation
-

02.0 The Transformation Pipeline

Learning Objectives:

- Name the five steps of the data transformation pipeline
- Explain what each step produces and why it is necessary
- Understand how the steps chain together to turn raw rows into a report

Content Outline:

- The pipeline: **Load** → **Filter** → **Group** → **Aggregate** → **Output**
- Load: read raw data from a file or database into a pandas DataFrame
- Filter: remove rows that don't belong in this report (wrong status, wrong date range)
- Group: organize remaining rows by a shared dimension (day, week, user)
- Aggregate: collapse each group into one number (count, sum, mean)
- Output: serialize the result as JSON or store it for later query
- How each step prepares the next — why skipping any step produces wrong results
- Section preview: 02.1 covers loading and filtering; 02.2 covers grouping and aggregating; 02.3 is the full pipeline challenge

Transitions: This overview frames all four lessons in Section 02. 02.1 starts with the first two steps.

Key Principles:

- Each step has one job — don't combine filtering logic inside a groupby
- The pipeline is deterministic: same input always produces the same output
- Filtering before grouping is always cheaper than filtering after

Examples:

- Load: `df = pd.read_csv('todos.csv')` → 1,000 rows
 - Filter: `df[df['status'] == 'completed']` → 340 rows
 - Group: `grouped = filtered.groupby(filtered['completed_at'].dt.date)` → 7 groups
 - Aggregate: `grouped['id'].count()` → 7 numbers
 - Output: `json.dumps(result.to_dict())`
-

02.1 Loading and Filtering

Learning Objectives:

- Load a CSV or database result into a pandas DataFrame
- Inspect a DataFrame to understand its structure before processing
- Apply boolean indexing to filter rows by status and date range

Content Outline:

- Reading data: `pd.read_csv()`, reading from a SQLAlchemy query result
- Inspecting: `df.dtypes`, `df.shape`, `df.head()` — know what you have before you filter
- Boolean indexing: `df[df['status'] == 'completed']`
- Date filtering: why string comparison on dates is unreliable
- Converting to datetime first: `df['completed_at'] = pd.to_datetime(df['completed_at'])`
- Building a date range filter: `df[df['completed_at'] >= cutoff]`
- Combining filters with `&`: `df[(condition_a) & (condition_b)]`
- Anti-pattern: filtering after groupby instead of before — wastes computation and produces wrong group sizes

Transitions: With clean, filtered data in the DataFrame, 02.2 shows how to group and aggregate it.

Key Principles:

- Always inspect before filtering — wrong assumptions about column names or types will silently produce empty results
- Convert to datetime before any date comparison — string comparison on ISO dates can appear to work but fails on edge cases
- Each filter reduces rows; applying them before groupby keeps all subsequent operations fast

Examples:

```
df = pd.read_csv('todos.csv')
df['completed_at'] = pd.to_datetime(df['completed_at'])
cutoff = datetime.now() - timedelta(days=7)
completed_this_week = df[(df['status'] == 'completed') & (df['completed_at']
>= cutoff)]
```

02.2 Grouping and Aggregating

Learning Objectives:

- Use `groupby()` to segment filtered data by a temporal dimension
- Extract date and week components from a datetime column
- Apply `.count()`, `.sum()`, and `.mean()` to reduce each group to a single value

Content Outline:

- `groupby()`: splits the DataFrame into groups sharing the same key
- Temporal dimensions: `df['completed_at'].dt.date` (by day), `.dt.isocalendar().week` (by week)
- Selecting the column to aggregate after groupby: `grouped['id'].count()`
- The three core aggregation functions:
 - `.count()`: how many rows (completions per day)
 - `.sum()`: total of a numeric column (total points earned per day)
 - `.mean()`: average of a numeric column (average time-to-complete per day)
- Chaining: `df.groupby(...)[ 'col' ].count()` returns a Series indexed by the group key
- Anti-pattern: calling `.count()` on the whole grouped DataFrame — counts ALL columns, including NaN-affected ones; always specify the column

Transitions: You now have a Series of one number per group. Section 03 covers what to do with that result.

Key Principles:

- `groupby()` doesn't compute anything — it sets up the grouping; the aggregation function does the computation
- Always name the column to aggregate explicitly: `grouped['id']` not just `grouped`
- The result of `groupby().count()` is indexed by the grouping key — that index becomes your report's dimension

Examples:

```
# Completions per day
daily =
completed_this_week.groupby(completed_this_week['completed_at'].dt.date)
['id'].count()
# daily is a Series: {date: count, date: count, ...}

# Average completion time per user
avg_time = df.groupby('user_id')['duration_minutes'].mean()
```

02.3 Build the Weekly Report 🖨️

Challenge Prompt: Given a CSV file of TODO events with columns `id`, `user_id`, `status`, and `completed_at`, write a Python script that filters for completed TODOs from the last 7 days, groups them by day, counts completions per day, and prints the result as JSON.

Solution Code:

```
import pandas as pd
import json
from datetime import datetime, timedelta

df = pd.read_csv('todos.csv')
df['completed_at'] = pd.to_datetime(df['completed_at'])

cutoff = datetime.now() - timedelta(days=7)
completed = df[
    (df['status'] == 'completed') &
    (df['completed_at'] >= cutoff)
]

daily_counts = completed.groupby(
    completed['completed_at'].dt.date
)['id'].count()

report = {str(k): int(v) for k, v in daily_counts.items()}
print(json.dumps(report, indent=2))
```

Challenge Code:

```
import pandas as pd
import json
from datetime import datetime, timedelta

df = pd.read_csv('todos.csv')
```

```
# Convert 'completed_at' to datetime
# your code here

# Define cutoff: 7 days ago from now
# your code here

# Filter: status == 'completed' AND completed_at >= cutoff
# your code here

# Group by day and count completions
# your code here

# Print result as JSON
# your code here
```

03.0 Report Outputs

Learning Objectives:

- Identify the two main options for delivering transformed data: serving and storing
- Explain the tradeoffs between computing on demand vs pre-computing
- Understand when each approach is appropriate

Content Outline:

- What you have: a pandas Series with one value per group
- Two paths: serve it immediately as a JSON API response, or persist it for later query
- Serve: compute at request time — fresh data, higher latency per request
- Store: compute ahead of time (via a scheduled job) — faster reads, data can go stale
- The question that decides: "How often is this report requested vs how often does the data change?"
- Section preview: 03.1 covers both approaches with code patterns

Transitions: This overview sets up the decision. 03.1 shows both patterns in code.

Key Principles:

- Neither approach is universally better — the choice depends on query frequency and acceptable staleness
- Most reports start as on-demand (serve) and become pre-computed (store) once request volume justifies it
- The transformation code is the same in both cases — only where and when it runs changes

Examples:

- Serve: dashboard makes a GET request → backend runs the pipeline → returns JSON in ~200ms
- Store: a nightly job runs the pipeline → writes results to a **reports** table → dashboard reads from that table in ~5ms

03.1 Serving vs Storing 📖

Learning Objectives:

- Convert a pandas Series to a dict or JSON string for API use
- Write a FastAPI endpoint that runs the transformation pipeline and returns the result
- Describe when pre-computing and storing report results makes sense

Content Outline:

- Serializing the result: `series.to_dict(), json.dumps({str(k): int(v) for k, v in series.items()})`
- Why date keys need `str()` conversion: JSON doesn't support `datetime.date` keys natively
- FastAPI pattern: route handler runs the pipeline and returns a `JSONResponse`
- Pre-computing and storing: save the aggregated result to a `reports` table or a JSON file
- Reading pre-computed reports: query the stored result instead of re-running the pipeline
- Deciding the threshold: if the report is requested hundreds of times per hour, pre-compute; if a few times per day, compute on demand
- Anti-pattern: serializing the raw DataFrame directly (`df.to_json()`) — returns all raw rows, not the aggregated report; always aggregate first

Transitions: You now have the complete mental model and toolkit for raw-to-report transformation. The assessment checks whether you can apply it to new scenarios.

Key Principles:

- Always aggregate before serializing — serving raw rows is not a report
- Date and datetime keys in dicts must be converted to strings before JSON serialization
- Pre-computing trades storage space and scheduling complexity for faster query time

Examples:

```
# FastAPI on-demand pattern
@app.get("/reports/weekly-completions")
def weekly_completions():
    df = pd.read_csv('todos.csv')
    df['completed_at'] = pd.to_datetime(df['completed_at'])
    cutoff = datetime.now() - timedelta(days=7)
    completed = df[(df['status'] == 'completed') & (df['completed_at'] >=
cutoff)]
    daily = completed.groupby(completed['completed_at'].dt.date)
    ['id'].count()
    return {str(k): int(v) for k, v in daily.items()}
```

04.0 Assessment 🧠

Learning Objectives:

- Apply the transformation pipeline to new data scenarios
- Identify correct and incorrect uses of filtering, grouping, and aggregation
- Choose the appropriate output strategy for a given use case

Question Format: Scenario-based (multiple-choice)

Questions:

1. Your `orders` table has columns `order_id`, `user_id`, `status`, `total_price`, and `created_at`. You want to know the total revenue per day for the last 30 days. Which pipeline is correct?
 - A) `df.groupby('created_at')['total_price'].sum()`
 - B) `df['created_at'] = pd.to_datetime(df['created_at']); cutoff = now - 30d; df[df['created_at'] >= cutoff].groupby(df['created_at'].dt.date)['total_price'].sum() ✓`
 - C) `df[df['status'] == 'paid'].groupby('user_id')['total_price'].mean()`
 - D) `df.groupby('user_id')['order_id'].count()`
2. After running `df.groupby(df['ts'].dt.date)['id'].count()`, the result is empty. The most likely cause is:
 - A) The `id` column is not numeric
 - B) The `ts` column was never converted to datetime ✓
 - C) `.count()` requires a numeric column
 - D) `groupby` only works with string columns
3. A dashboard requests the weekly engagement report once per minute. The underlying event data updates once per hour. The best output strategy is:
 - A) Compute on demand — run the pipeline on every request
 - B) Pre-compute hourly and serve the stored result ✓
 - C) Serve raw rows and let the frontend aggregate
 - D) Cache the result in memory indefinitely
4. You want to find the average number of sessions per user per week. Which aggregation function do you use after `groupby(['user_id', week_dimension])`?
 - A) `.count()`
 - B) `.sum()`
 - C) `.mean()` ✓
 - D) `.max()`
5. Your FastAPI endpoint calls `return df.to_json()` directly after loading the CSV. What is wrong with this?
 - A) FastAPI can't return a JSON string
 - B) `.to_json()` only works on Series, not DataFrames
 - C) It returns all raw rows instead of an aggregated report ✓
 - D) Nothing — this is a valid pattern

6. You filter with `df[df['date'] >= '2024-01-01']` without converting the column to datetime first. The risk is:
- A) The filter always raises an error
 - B) String comparison can silently produce incorrect results for dates near month or year boundaries ✓
 - C) pandas always converts strings to datetime automatically
 - D) The filter only works on numeric columns
7. You need to build a report showing completions per week (not per day). After converting `completed_at` to datetime, which expression gives you the week dimension for groupby?
- A) `df['completed_at'].dt.date`
 - B) `df['completed_at'].dt.isocalendar().week` ✓
 - C) `df['completed_at'].dt.month`
 - D) `df['completed_at'].dt.weekday`
8. You call `df.groupby('date').count()` without specifying a column. The risk is:
- A) It raises an `AttributeError`
 - B) It counts all columns, including ones with NaN that would give misleading counts ✓
 - C) It only counts the first column
 - D) It returns the sum instead of the count

Evaluation Criteria:

- Correctly identifies the full filter → group → aggregate → output sequence
- Understands datetime conversion as a prerequisite for temporal filtering
- Selects the right aggregation function for each metric type
- Chooses the correct output strategy based on query frequency

Score Interpretation:

- 8/8: Ready to build production data transformation pipelines
- 6–7: Solid grasp; review the lessons for any missed questions
- 4–5: Re-read Sections 02 and 03 before moving on
- Below 4: Restart from Section 01

05.0 Data Into Decisions

Content Outline:

- Course recap: what students accomplished
 - Named the gap between CRUD data and analytics
 - Built the complete 5-step pipeline (load → filter → group → aggregate → output)
 - Wrote a full working weekly report script in pandas
 - Learned when to serve on demand vs pre-compute and store
- Connection to bigger picture: this pipeline is the foundation of every analytics feature — dashboards, trend lines, product metrics, ML feature engineering

- Next steps: Skill 45 covers scheduling this pipeline to run automatically; the combination of transformation + scheduling = automated reports
- Resources for further exploration:
 - pandas documentation: `groupby`, `resample`, `pivot_table`
 - "Python for Data Analysis" (Wes McKinney)
 - Real-world practice: apply this to any dataset you already have
- Encouragement: the hardest part was naming the problem. The pipeline itself is just five lines — you now have a mental model that works on any dataset.

Note: This is conclusion only — no theory, exercises, or assessment.